

Plano de Implementacao

Scraping TJSP - Jurisprudencia CJSG

Versao 1.0

17/08/2026

Projeto: scraping_tjsp v0.2.0

Documento gerado automaticamente. Nao reflete alteracoes no codigo base.

Plano de Implementação - Scraping TJSP

Versão: 1.0

Data: 17/08/2026

Projeto: scraping_tjsp v0.2.0

Escopo: Melhorias sugeridas (sem alterações no código existente)

Sumário

1. Arquitetura & Código
2. Persistência & Dados
3. API & Web
4. Testes & Qualidade
5. Observabilidade
6. Performance & Escalabilidade
7. Funcionalidades
8. Empacotamento & DX
9. Priorização & Cronograma

1. Arquitetura & Código

1.1 Separação de Responsabilidades (PipelineColeta)

Arquivo: src/scraping_tjsp/cli.py:125-226

Problema: Função main() orquestra busca, persistência, download, processamento e saída.

Solução: Criar PipelineColeta com etapas independentes:

- EtapaBusca → EtapaPersistencia → EtapaDownload → EtapaProcessamento → EtapaSaida
- Cada etapa testável isoladamente, com Protocol para injeção.

1.2 Configuração Centralizada (settings.py)

Arquivos: api.py:48-99, cli.py:56-121

Problema: Defaults duplicados entre CLI e API.

Solução: pydantic-settings com BaseSettings:

```
class Settings(BaseSettings):
    sqlite_path: Path = Path("data/tjsp.sqlite3")
    chroma_path: Path = Path("data/chroma")
    intervalo_tjsp: float = 2.0
    max_pdfs: int = 20
```

```
maritaca_api_key: str | None = None
# ...
class Config:
    env_file = ".env"
    env_prefix = "TJSP_"
```

1.3 Token Bucket Rate Limiter

Arquivo: src/scraping_tjsp/client.py:17-27

Problema: _Limitador usa fixed window (pode permitir burst no limite).

Solução: Implementar token bucket com asyncio/threading:

```
class TokenBucket:
    def __init__(self, rate: float, burst: int):
        self.rate = rate
        self.burst = burst
        self._tokens = burst
        self._last = time.monotonic()
```

1.4 Retry Expandido no HTTP Client

Arquivo: src/scraping_tjsp/client.py:108-115

Problema: Retry só em status codes, não em exceções de conexão.

Solução: Adicionar connect=3, read=3, status=3 com Retry do urllib3 + jitter.

2. Persistência & Dados

2.1 Migrações SQLite Versionadas

Arquivo: src/scraping_tjsp/storage.py:29-37

Problema: executscript(schema) não versiona.

Solução: Tabela schema_version + diretório migrations/:

```
CREATE TABLE schema_version (version INTEGER PRIMARY KEY, applied_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
```

Scripts: 001_initial.sql, 002_add_arquivado.sql, etc.

2.2 Índices Compostos FTS5

Arquivo: src/scraping_tjsp/storage.py:510-606

Problema: JOIN pesado em buscar_chunks_lexical.

Solução: Criar view materializada ou índices:

```
CREATE INDEX idx_chunks_fts_acordao ON chunks_documento(cd_acordao);
CREATE INDEX idx_chunks_fts_processo ON chunks_documento(processo);
```

2.3 Versionamento de Coleção Chroma

Arquivo: src/scraping_tjsp/vector_store.py

Problema: Não valida modelo/dimensão ao inicializar.

Solução: Metadata na coleção:

```
collection.metadata = {
    "embedding_model": "all-MiniLM-L6-v2",
    "dimension": 384,
    "created_at": "2026-08-17"
}
```

Validar no `__init__` e rejeitar se divergir.

2.4 Soft Delete / Arquivamento

Tabela: documentos

Solução: Adicionar status arquivado + coluna arquivado_em. CLI `tjsp-arquivar` para limpeza.

3. API & Web

3.1 Autenticação API Key

Arquivo: `src/scraping_tjsp/api.py`

Solução: Middleware simples:

```
API_KEYS = {"default": os.environ.get("TJSP_API_KEY", "")}

async def verify_api_key(x_api_key: str = Header(...)):
    if x_api_key not in API_KEYS.values():
        raise HTTPException(401, "Invalid API key")
```

Aplicar em endpoints de custo (`/perguntar`, `/tjsp/pesquisa-assistida`, `/tjsp/analisar-documentos`).

3.2 Rate Limiting na API

Dependência: `slowapi`

Solução:

```
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

@app.post("/perguntar")
@limiter.limit("10/minute")
async def perguntar(...):
```

3.3 SSE para Operações Longas

Endpoints: `/tjsp/pesquisa-assistida`, `/tjsp/analisar-documental`

Solução: Retornar 202 Accepted + `task_id`, expor `/tasks/{id}/stream` (SSE) com progresso:

```
{"stage": "planejamento", "progress": 10, "message": "Gerando consultas..."}
{"stage": "coleta", "progress": 40, "message": "Consultando TJSP..."}
{"stage": "ranking", "progress": 80, "message": "Rankeando resultados..."}
{"stage": "concluido", "progress": 100, "result": {...}}
```

3.4 OpenAPI Enriquecido

Arquivo: src/scraping_tjsp/api.py

Solução: Adicionar summary, description, response_examples em cada endpoint.

3.5 Security Headers

Middleware:

```
@app.middleware("http")
async def security_headers(request, call_next):
    response = await call_next(request)
    response.headers["Content-Security-Policy"] = "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'"
    response.headers["X-Frame-Options"] = "DENY"
    response.headers["Referrer-Policy"] = "strict-origin-when-cross-origin"
    return response
```

4. Testes & Qualidade

4.1 Estrutura de Testes

```
tests/
├── unit/
│   ├── test_parser.py
│   ├── test_models.py
│   ├── test_search.py
│   └── test_storage.py
├── integration/
│   ├── test_api.py
│   └── test_cli.py
├── contract/
│   └── test_schemas.py
├── fixtures/
│   └── sample_html/
```

4.2 Property-Based Testing (Hypothesis)

Alvos: parser.py, models.py, search.py

Exemplo:

```
@given(st.text(min_size=1, max_size=200))
def test_rrf_fusion_commutative(texto):
    # RRF(semantic, lexical) == RRF(lexical, semantic) quando pesos iguais
```

4.3 Mutation Testing

Ferramenta: mutmut

Config: pyproject.toml

```
[tool.mutmut]
paths_to_mutate = "src/scraping_tjsp"
backup = False
runner = "python -m pytest -x"
```

4.4 Benchmark de Busca

Script: scripts/benchmark_busca.py

Métricas: latência p50/p95/p99, throughput, recall@k para BM25 vs Vetorial vs Híbrida.

5. Observabilidade

5.1 Structured Logging (structlog)

Substitui: print() no CLI, logging básico na API.

Config:

```
import structlog

structlog.configure(
    processors=[
        structlog.processors.add_request_id,
        structlog.processors.TimeStamper(fmt="iso"),
        structlog.processors.JSONRenderer()
    ]
)
logger = structlog.get_logger()
```

5.2 Métricas Prometheus

Endpoint: /metrics

Métricas:

```
# HELP http_requests_total Total HTTP requests
# TYPE http_requests_total counter
http_requests_total{method="POST",endpoint="/perguntar",status="200"} 142

# HELP tjsp_request_duration_seconds TJSP request latency
# TYPE tjsp_request_duration_seconds histogram
tjsp_request_duration_seconds_bucket{le="1.0"} 89
tjsp_request_duration_seconds_bucket{le="2.0"} 134

# HELP maritaca_tokens_total Tokens consumidos
# TYPE maritaca_tokens_total counter
maritaca_tokens_total{direction="input"} 450000
maritaca_tokens_total{direction="output"} 120000
```

5.3 Distributed Tracing (OpenTelemetry)

Instrumentação: opentelemetry-instrument + opentelemetry-exporter-otlp

Spans: CLI → API → Maritaca → TJSP com trace_id propagado.

5.4 Health Checks Profundos

Endpoint: GET /saude (expandir api.py:249-267)

```
{
  "status": "ok",
  "checks": {
```

```
"sqlite": {"status": "ok", "writable": true, "path": "data/tjsp.sqlite3"},
"chroma": {"status": "ok", "collections": 2, "vectors": 15420},
"maritaca": {"status": "ok", "key_configured": true},
"tesseract": {"status": "ok", "version": "5.3.0", "languages": ["por"]}
}
```

6. Performance & Escalabilidade

6.1 Connection Pool SQLite

Problema: sqlite3.connect por request na API.

Solução: Pool com queue.Queue + check_same_thread=False:

```
class SQLitePool:
    def __init__(self, path: Path, size: int = 10):
        self._pool = queue.Queue(maxsize=size)
        for _ in range(size):
            conn = sqlite3.connect(path, check_same_thread=False)
            conn.execute("PRAGMA busy_timeout = 30000")
            self._pool.put(conn)

    @contextmanager
    def connect(self):
        conn = self._pool.get()
        try:
            yield conn
        finally:
            self._pool.put(conn)
```

6.2 Batch Chroma Upsert

Arquivo: src/scraping_tjsp/vector_store.py

Solução: Acumular em buffer e add em lote:

```
def indexar_lote(self, documentos: list[DocumentoComEmbedding]):
    ids = [d.id for d in documentos]
    embeddings = [d.embedding for d in documentos]
    metadatas = [d.metadata for d in documentos]
    texts = [d.texto for d in documentos]
    self.collection.add(ids=ids, embeddings=embeddings, metadatas=metadatas, documents=texts)
```

6.3 Streaming PDF Download

Arquivo: src/scraping_tjsp/downloader.py

Solução: Gravar direto em temp file:

```
with tempfile.NamedTemporaryFile(delete=False, dir=self.diretorio, suffix=".pdf") as tmp:
    for chunk in response.iter_content(chunk_size=8192):
        tmp.write(chunk)
    tmp.flush()
    os.fsync(tmp.fileno())
shutil.move(tmp.name, destino_final)
```

6.4 Async HTTP Client

Dependência: httpx (já em dev deps)

Migração: TJSPClient → AsyncTJSPClient para API + pesquisa assistida.

6.5 Cache de Embeddings

Estrutura: LRU disk cache keyed by cd_acordao:model_version

Biblioteca: diskcache ou joblib.Memory

7. Funcionalidades

7.1 Export/Import Dataset

CLI: tjsp-exportar, tjsp-importar

Formato: JSONL com consultas, decisoes, chunks, documentos.

7.2 Deduplicação Semântica

Algoritmo: SimHash (64-bit) ou MinHash LSH

Aplicação: Na indexação, detectar near-duplicates (threshold 0.9) e fundir ou marcar duplicado_de.

7.3 Scheduler de Coletas

Biblioteca: apscheduler

Config: YAML com jobs recorrentes:

```
jobs:
- nome: "diario_dano_moral"
  cron: "0 6 * * *"
  consulta:
    pesquisa: "dano moral"
    inicio: "-1d"
    fim: "hoje"
    acoes: ["baixar-pdfs", "processar-pdfs"]
```

7.4 Multi-Modelo Embedding UI

CLI: tjsp-preparar-dataset --embedding-model all-MiniLM-L6-v2|BAAI/bge-m3|intfloat/e5-large

Automático: Cria coleção separada por modelo (chroma-minilm, chroma-bge-m3, etc.)

7.5 Validação Jurídica Expandida

Arquivo: src/scraping_tjsp/legal_validation.py (atual: 3128 bytes)

Adicionar:

- Validação citações leis (ex: Lei 8.078/90, CPC/2015 art. 319)
- Súmulas STJ/STF (regex + base local)
- Precedentes vinculantes STF
- Normas regulamentares (CNCJ, CNMP)

8. Empacotamento & DX

8.1 Publicação PyPI (Trusted Publishing)

GitHub Actions: `.github/workflows/publish.yml`

```
name: Publish to PyPI
on:
  release:
    types: [published]
permissions:
  id-token: write
jobs:
  publish:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pypa/gh-action-pypi-publish@release/v1
```

8.2 Dockerfile Multi-Stage

```
# Builder
FROM python:3.11-slim AS builder
WORKDIR /app
COPY pyproject.toml README.md ./
RUN pip install --no-cache-dir build && python -m build --wheel

# Runtime
FROM python:3.11-slim
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends \
  tesseract-ocr tesseract-ocr-por poppler-utils \
  && rm -rf /var/lib/apt/lists/*
COPY --from=builder /app/dist/*.whl ./
RUN pip install --no-cache-dir ./*.whl
USER 1000:1000
ENTRYPOINT ["tjsp-api"]
```

8.3 Pre-commit Hooks

Arquivo: `.pre-commit-config.yaml`

```
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.12.0
    hooks:
      - id: ruff-check
      - id: ruff-format
  - repo: https://github.com/pre-commit/mirrors-mypy
    rev: v1.14.0
    hooks:
      - id: mypy
        additional_dependencies: [types-requests]
  - repo: local
    hooks:
      - id: pytest-check
        name: pytest (fast)
        entry: python -m pytest -x --co -q
        language: system
        pass_filenames: false
```

8.4 DevContainer

Diretório: .devcontainer/

```
{
  "name": "scraping-tjsp",
  "image": "mcr.microsoft.com/devcontainers/python:3.11",
  "features": {
    "ghcr.io/devcontainers/features/docker-in-docker:2": {}
  },
  "postCreateCommand": "powershell -ExecutionPolicy Bypass -File scripts/bootstrap-dev.ps1",
  "customizations": {
    "vscode": {
      "extensions": ["ms-python.python", "ms-python.vscode-pylance", "charliermarsh.ruff"]
    }
  }
}
```

9. Priorização & Cronograma

Matriz de Priorização (Impacto × Esforço)

Iniciativa	Impacto	Esforço	Prioridade	Sprint Sugerida
Config centralizada (settings.py)	Alto	Baixo	**P0**	1
Migrações SQLite	Alto	Baixo	**P0**	1
Auth API Key	Alto	Médio	**P0**	1
Rate limiting API	Alto	Médio	**P0**	2
Structured logging	Médio	Baixo	**P1**	2
Métricas Prometheus	Médio	Médio	**P1**	3
Connection pool SQLite	Alto	Médio	**P1**	3
Batch Chroma upsert	Médio	Baixo	**P1**	3
Async HTTP client	Alto	Alto	**P2**	4
SSE para ops longas	Médio	Médio	**P2**	4
Property-based tests	Médio	Médio	**P2**	5
Mutation testing	Baixo	Médio	**P3**	5
Export/Import dataset	Médio	Médio	**P2**	6
Scheduler coletas	Médio	Alto	**P3**	6
Multi-modelo embedding	Baixo	Médio	**P3**	7
Dockerfile + PyPI publish	Médio	Baixo	**P1**	2
DevContainer + pre-commit	Baixo	Baixo	**P2**	3

Estimativa de Esforço Total

- P0 (Crítico): ~40h
- P1 (Alto): ~60h
- P2 (Médio): ~80h
- P3 (Baixo): ~40h
- Total: ~220h (~6 semanas @ 1 dev)

Apêndice A: Arquivos-Chave Referenciados

Arquivo	Linhas	Descrição
`src/scraping_tjsp/cli.py`	125-226	Pipeline principal de coleta
`src/scraping_tjsp/client.py`	17-27, 108-115	Rate limiter + HTTP retry
`src/scraping_tjsp/storage.py`	29-37, 510-606	Schema init + busca lexical
`src/scraping_tjsp/vector_store.py`	-	Chroma operations
`src/scraping_tjsp/api.py`	48-99, 183-236, 249-267	Config API + app factory + health
`src/scraping_tjsp/search.py`	25-93	Busca híbrida RRF
`src/scraping_tjsp/downloader.py`	-	PDF download
`src/scraping_tjsp/legal_validation.py`	-	Validação jurídica (mínima)
`src/scraping_tjsp/web/app.js`	-	Frontend SPA
`pyproject.toml`	-	Deps + scripts + config tools
`schema_sqlite.sql`	-	Schema SQLite

Apêndice B: Dependências Novas Sugeridas

```
# pyproject.toml [project.optional-dependencies]
observability = [
    "structlog>=24,<25",
    "prometheus-client>=0.19,<1",
    "opentelemetry-api>=1.22,<2",
    "opentelemetry-sdk>=1.22,<2",
    "opentelemetry-exporter-otlp>=1.22,<2",
    "opentelemetry-instrumentation-fastapi>=0.42,<1",
    "opentelemetry-instrumentation-requests>=0.42,<1",
]
api = [
    "slowapi>=0.1,<1",
    "python-jose>=3,<4", # se JWT/OAuth2
]
testing = [
    "hypothesis>=6,<7",
    "mutmut>=2,<3",
    "pytest-benchmark>=4,<5",
]
dx = [
    "pre-commit>=3,<4",
]
scheduler = [
    "apscheduler>=3,<4",
]
embeddings = [
    "diskcache>=5,<6",
    "sentence-transformers>=5.6,<6",
]
```

Documento gerado automaticamente em 17/08/2026. Não reflete alterações no código base.